F1 Deep Audit — Everything I Need to Know as an AI PM
Feature: F1 — Regulatory Feed Monitoring
Status: Complete
Audited: 2026-06-01
Author: RegWatch AI Build Session

This is my permanent reference document for F1. Written by reading every line of every file in the codebase — not from memory.

SECTION 1: Project File Map
FILE: src/models.py
DOES: Defines all three database tables as Python classes — Agency, RegulatoryDocument, AuditLog. Every other file in the project imports from here.
KEY CLASS: RegulatoryDocument — the atomic unit of the entire product. Every feature F1–F5 reads from or writes to this table.
CONNECTS TO: Every file in the project imports at least one class from here. database.py uses it to create tables. f1_ingest/*.py all import RegulatoryDocument. dashboard/app.py imports DocType, SourceAgency, RegulatoryDocument.
BREAKS IF DELETED: Everything. The entire project stops working. No tables exist, no imports succeed, no pipeline runs.
WHY THIS WAY: SQLModel merges SQLAlchemy (DB engine) and Pydantic (validation) into one class. The alternative — two separate class definitions per table — creates drift risk where the DB schema and the Python schema silently disagree. One file, one definition, one source of truth.

FILE: src/database.py
DOES: Creates the database engine, provides get_session() context manager, and exposes create_db_and_tables() to initialise the schema.
KEY FUNCTION: get_session() — a context manager that yields a DB session and guarantees cleanup even if an exception occurs mid-operation.
CONNECTS TO: Imported by every file that touches the database: agencies.py, dedup.py, anomaly.py, fulltext.py, ingest.py, health.py, query.py, dashboard/app.py.
BREAKS IF DELETED: All database operations fail. No reads, no writes, no session management.
WHY THIS WAY: Separating connection management from model definitions means models can be imported in tests without creating a live DB connection. The DATABASE_URL comes from .env — swapping SQLite for Postgres requires only an environment variable change, zero code changes.

FILE: src/f1_ingest/agencies.py
DOES: Defines the 6 agency feed configurations as Python dicts and provides seed_agencies() to write them to the DB on first run.
KEY FUNCTION: seed_agencies() — idempotent insert: checks slug uniqueness before inserting, safe to run multiple times.
CONNECTS TO: scripts/setup_db.py calls seed_agencies(). ingest.py uses FR_API_SLUGS to route each agency to the correct fetcher.
BREAKS IF DELETED: setup_db.py fails. The Agency table stays empty. The ingest pipeline finds no active agencies and exits immediately.
WHY THIS WAY: Alternative was a YAML/JSON config file. Storing in DB means a future UI can let Sarah toggle agencies on/off without a code deployment. active=False disables a feed without deleting the row — history preserved.

FILE: src/f1_ingest/fetcher.py
DOES: Two fetchers — fetch_feed() for RSS (Federal Reserve), fetch_fr_api() for the Federal Register JSON API (CFPB, OCC, FDIC, FinCEN). Both return lists of unsaved RegulatoryDocument objects.
KEY FUNCTION: fetch_fr_api() — handles the 4 agencies whose RSS feeds block automated requests. Uses the FR public JSON API instead.
CONNECTS TO: Called by ingest.py. Imports classifier.py (to classify each document), dedup.py (to compute hash), models.py (to construct documents).
BREAKS IF DELETED: The ingest pipeline cannot fetch any documents. 0 documents ingested per run.
WHY THIS WAY: We discovered on Day 2 that the Federal Register RSS feeds return an HTML "Request Access" page (status 200 but no XML) when accessed programmatically. Their JSON API is public and returns clean structured data. Using two fetchers (one per source type) keeps each fetcher simple and focused.

FILE: src/f1_ingest/classifier.py
DOES: Keyword-matches a document title against ordered rule lists and returns a DocType enum value (Final Rule, Proposed Rule, Guidance, Enforcement, FAQ, Other).
KEY FUNCTION: classify_doc_type(title: str) -> DocType — loops through _RULES in priority order, returns first match, falls back to OTHER.
CONNECTS TO: Called inside fetch_feed() and fetch_fr_api() in fetcher.py for every

document parsed. Also tested directly in tests/test_f1_classifier.py.
BREAKS IF DELETED: Every document is ingested as DocType.OTHER. Dashboard loses doc type filtering. F4 task prioritisation (which depends on doc type) breaks in Week 5.
WHY THIS WAY: Rule-based over ML because: (1) runs on every document, zero cost, zero latency; (2) explainable — you can see exactly why a document was classified; (3) regulatory titles use predictable vocabulary. Accuracy is ~6% (7/111 classified non-Other) — low, but F2 LLM will reclassify with full document context.

FILE: src/f1_ingest/dedup.py
DOES: Computes SHA-256(title + url) as a content fingerprint and checks whether that hash already exists in the database.
KEY FUNCTION: is_duplicate(content_hash: str) -> bool — single DB lookup, returns True/False.
CONNECTS TO: Called in ingest.py for every document before saving. compute_hash() is called inside fetcher.py when constructing each document.
BREAKS IF DELETED: Every document is saved on every run, including re-runs. Joint agency rules appear multiple times. F2 summarises the same document repeatedly. AuditLog becomes meaningless noise.
WHY THIS WAY: SHA-256 on title+url handles two real failure modes: (1) same document at different URLs (cross-posted joint rules) — caught because title is the same; (2) same URL with different titles — caught because URL alone wouldn't be enough. Proved on Day 2: 9 cross-feed duplicates correctly caught on first run.

FILE: src/f1_ingest/fulltext.py
DOES: Fetches the complete regulation text for each document. Uses FR API raw_text_url for Federal Register docs (plain text), and BeautifulSoup HTML parsing for Fed press releases. Also contains title similarity deduplication logic.
KEY FUNCTION: run_fulltext_enrichment(limit: int) — batch enriches documents with short/missing raw_content, oldest first, with 1-second rate limiting.
CONNECTS TO: Called in ingest.py after each agency's new documents are saved. Also called directly by scripts/enrich_fulltext.py.
BREAKS IF DELETED: raw_content stays as 1–2 sentence abstracts. F2 summarises abstracts instead of full regulations. Summary quality collapses. The title_similarity() and find_near_duplicates() functions also disappear — near-duplicate detection is lost.
WHY THIS WAY: Two strategies because two different source types: FR documents have a clean raw_text_url endpoint (no parsing needed); Fed press releases are HTML pages requiring BeautifulSoup. Using the API endpoint when available is always preferred over screen-scraping.

FILE: src/f1_ingest/anomaly.py
DOES: Runs two anomaly detectors on newly ingested documents — Z-score volume spike detection and off-schedule day-of-week detection. Writes is_anomaly=True to flagged documents and logs to AuditLog.
KEY FUNCTION: run_anomaly_check(new_docs: list[RegulatoryDocument]) -> int — orchestrates both detectors, returns count of flagged documents.
CONNECTS TO: Called in ingest.py after documents are saved and enriched. Writes to RegulatoryDocument.is_anomaly and AuditLog. Dashboard reads is_anomaly to surface the red alert banner.
BREAKS IF DELETED: No anomaly detection. is_anomaly stays False on all documents. Dashboard anomaly banner never appears. Sarah never gets proactive alerts about unusual publication spikes.
WHY THIS WAY: Z-score over Isolation Forest because: (1) we have 1 day of history — Isolation Forest needs weeks to train meaningfully; (2) Z-score output is explainable ("published 3x its 30-day average") while Isolation Forest gives an opaque score; (3) in a compliance product, explainability to regulators matters more than marginal accuracy gains.

FILE: src/f1_ingest/ingest.py
DOES: The pipeline orchestrator. Loads active agencies, routes to correct fetcher, deduplicates, saves, enriches with full text, runs anomaly detection, writes AuditLog.
KEY FUNCTION: run_ingest(agency_slugs=None) -> dict — runs the complete pipeline for all active agencies (or a subset), returns summary dict.
CONNECTS TO: Imports and calls every other F1 component. Called by scripts/run_daily.py, scripts/daily_validate.py, and directly via python -m src.f1_ingest.ingest.
BREAKS IF DELETED: The pipeline cannot be run. All individual components exist but nothing coordinates them.
WHY THIS WAY: Separating orchestration (ingest.py) from data fetching (fetcher.py), classification (classifier.py), and deduplication (dedup.py) means each component can be tested independently. ingest.py has no business logic — it only coordinates.

FILE: src/f1_ingest/health.py

DOES: Two read-only checks per agency — reachability (can we reach the feed and get ≥1 entry?) and freshness (do we have DB documents from this agency within 3 days?).
KEY CLASS: AgencyHealth — dataclass with healthy, status_label, last_doc_date properties.
CONNECTS TO: Called by scripts/run_daily.py and scripts/daily_validate.py. Does not write to DB.
BREAKS IF DELETED: No feed monitoring. Silent failures go undetected. If a government site changes its URL, the system ingests 0 documents and nobody knows until Sarah reports missing regulations.
WHY THIS WAY: Two checks instead of one because a feed can be reachable but stale (returns 200 with zero content — exactly what the Federal Register RSS feeds did when they returned an HTML gate page). Reachability alone would have passed. Freshness catches the silent failure.

FILE: src/f1_ingest/query.py
DOES: CLI tool to inspect database contents — summary stats by agency and doc type, recent documents list, anomaly-flagged documents view.
KEY FUNCTION: show_summary() — prints total documents, breakdown by agency and type, anomaly/review counts.
CONNECTS TO: Reads RegulatoryDocument from DB. No writes. Standalone — not called by any other file.
BREAKS IF DELETED: No CLI inspection tool. Developers and PMs must use DB Browser for SQLite to inspect data. Annoying but not a pipeline failure.
WHY THIS WAY: Alternative was waiting until the Streamlit dashboard existed. The query tool gives immediate visibility during development and becomes useful again when the server isn't running.

FILE: dashboard/app.py
DOES: The Streamlit browser dashboard. Loads all documents from DB, applies sidebar filters (agency, doc type, anomaly), sorts, and renders expandable document cards.
KEY FUNCTION: load_documents() — cached DB query (5-minute TTL) that returns documents as plain dicts for Streamlit serialisation.
CONNECTS TO: Imports database.py, models.py, dashboard/components.py. Reads RegulatoryDocument. No writes.
BREAKS IF DELETED: No browser UI. Week 1 exit gate unmet — Mike cannot view filtered feeds.
WHY THIS WAY: Streamlit over React because React requires npm, a build pipeline, component libraries, and a FastAPI backend — weeks of work. Streamlit is pure Python, browser UI in one file, deployable with one command. Correct tool for a pilot demo. React comes in Week 6 with the full API layer.

FILE: dashboard/components.py
DOES: Reusable UI helpers — colour-coded doc type badges, anomaly badge, KPI metric row, expandable document card renderer, sidebar filter controls.
KEY FUNCTION: render_document_card(doc) — renders one document as an expandable Streamlit expander with badges, content preview, and source link.
CONNECTS TO: Imported by dashboard/app.py. No other dependencies.
BREAKS IF DELETED: app.py cannot render anything — all display imports fail.
WHY THIS WAY: Separating display logic from data logic means when React replaces Streamlit in Week 6, only components.py changes — the data queries and filter logic in app.py stay the same.

FILE: scripts/setup_db.py
DOES: One-time database initialisation — creates all tables, seeds all 6 agencies. Safe to run multiple times.
KEY FUNCTION: Calls create_db_and_tables() then seed_agencies().
CONNECTS TO: database.py, agencies.py.
BREAKS IF DELETED: New developers cannot set up the project. No documentation on how to initialise the DB.
WHY THIS WAY: Idempotent by design — running it twice doesn't create duplicate agencies. This is the only correct way to write setup scripts.

FILE: scripts/run_daily.py
DOES: The daily pipeline entry point — runs health check, ingest, and full-text enrichment in sequence, logs everything to logs/daily_run.log.
KEY FUNCTION: main() -> int — returns 0 on success, 1 on unhealthy feeds. Exit code checked by Task Scheduler.
CONNECTS TO: health.py, ingest.py, fulltext.py.
BREAKS IF DELETED: Task Scheduler has nothing to call. Daily automation stops. Manual runs still work via python -m src.f1_ingest.ingest.
WHY THIS WAY: Stable entry point that Task Scheduler calls forever. Adding F2 summarisation to the daily run (Week 2) means editing this file — not the Task Scheduler config.

FILE: scripts/schedule_daily.py
DOES: Creates, removes, or checks the Windows Task Scheduler task "RegWatch-AI-Daily".
KEY FUNCTION: register_task(run_time) — calls schtasks /Create with the correct Python path and script path.
CONNECTS TO: Nothing in src/. Standalone Windows automation script.
BREAKS IF DELETED: Cannot register or manage the scheduled task via command line. Task Scheduler UI still works manually.
WHY THIS WAY: Windows Task Scheduler over cron/APScheduler because it's built into Windows, survives reboots without a background process, and is visible in the Windows UI.

FILE: fixtures/golden/f1_golden_set.json
DOES: 10 hand-labeled regulatory documents with ground-truth fields (doc_type, headline, what_changed, effective_date, affected_institution_types, confidence_floor). The acceptance test for F2.
KEY FIELD: eval_instructions — defines the exact criteria: faithfulness (no invented facts), relevance (answers compliance officer's question), passing threshold (RAGAS faithfulness ≥ 0.85, answer_relevance ≥ 0.80).
CONNECTS TO: Will be read by the F2 eval harness (built in Week 3). Currently standalone.
BREAKS IF DELETED: F2 has no ground truth to evaluate against. We cannot objectively say F2 is "done."
WHY THIS WAY: Hand-labeled by reading actual document content — not generated by an LLM. If Claude generates the labels and Claude produces the summaries, we measure self-consistency, not accuracy.

FILE: docs/FP-FN-Risk-Matrix.md
DOES: Quantifies the business cost of false negatives (missed regulations → $500K–$5M fines) vs false positives (false alarms → wasted time, trust erosion) for both anomaly detection and F2 summarisation.
KEY SECTION: "Asymmetry Summary" — the table showing FN costs millions while FP costs 30 minutes. This asymmetry drives every threshold decision.
CONNECTS TO: Referenced in design decisions for anomaly threshold (Z=2.0), confidence threshold (0.80), and HITL gate (F4). Not imported by code.
BREAKS IF DELETED: Design decisions lose their written rationale. Future threshold changes are made without understanding the cost asymmetry.
WHY THIS WAY: Written before F2 is built — eval-first principle. Defining failure costs before building the system ensures thresholds are set for the right reasons.

SECTION 2: Data Flow — One Document End to End
Document chosen: "Federal Reserve Board issues enforcement actions with former employee of Atlantic Union Bank and former employee of Frost Bank" (May 28, 2026)

STEP 1: Entry — where does it enter the system?

File: scripts/run_daily.py → calls run_ingest() → ingest.py line 47

fetcher = fetch_fr_api if agency.slug in FR_API_SLUGS else fetch_feed
documents = fetcher(agency)
The Federal Reserve slug "fed" is NOT in FR_API_SLUGS, so fetch_feed() is called.

Inside fetcher.py, fetch_feed() (line 113):

Makes an HTTP GET to https://www.federalreserve.gov/feeds/press_all.xml
Passes the response text to feedparser.parse()
Loops through feed.entries
For this entry: title = "Federal Reserve Board issues enforcement actions...", url = "https://www.federalreserve.gov/newsevents/pressreleases/enforcement20260528a.htm"
Still inside fetch_feed(), for each entry:

compute_hash(title, url) → dedup.py line 16 → SHA-256 of concatenated strings → 64-char hex string
classify_doc_type(title) → classifier.py line 59 → scans title for keywords → finds "enforcement action" → returns DocType.ENFORCEMENT
_parse_date(entry) → extracts published_parsed from feedparser → returns datetime(2026, 5, 28, ...)
raw_content = entry.summary (short abstract from RSS)
A RegulatoryDocument object is constructed (not yet saved) and appended to documents.

STEP 2: Deduplication — what check runs?

Back in ingest.py line 58:

```
if is_duplicate(doc.content_hash):
    duplicate_count += 1
    continue
is_duplicate() in dedup.py line 22:

existing = session.exec(
    select(RegulatoryDocument).where(
        RegulatoryDocument.content_hash == content_hash
    )
).first()
return existing is not None
Runs a SQL query: SELECT * FROM regulatorydocument WHERE content_hash = '<hash>' LIMIT 1

If duplicate: duplicate_count += 1, document is discarded, loop continues to next entry. Nothing written to DB. If new: proceeds to save.

STEP 3: Saving to database

ingest.py lines 62-67:

with get_session() as session:
    session.add(doc)
    session.commit()
    session.refresh(doc)
    new_count += 1
    saved_docs.append(doc)
Fields written to regulatorydocument table:

id = auto-generated UUID (e.g. "cfd1ed81-8384-4eae-b394-b766ed4348cd")
source_agency = "fed"
doc_type = "enforcement"
title = "Federal Reserve Board issues enforcement actions..."
url = "https://www.federalreserve.gov/.../enforcement20260528a.htm"
published_date = 2026-05-28 11:00:00
fetched_at = current UTC datetime
content_hash = SHA-256 hex string
raw_content = short RSS abstract (~200 chars)
summary_json = None (F2 hasn't run yet)
status = "new"
review_flag = False
is_anomaly = False (anomaly check hasn't run yet)
STEP 4: Full-text enrichment

ingest.py line 71: run_fulltext_enrichment(limit=len(saved_docs))

Inside fulltext.py, enrich_document(doc) line 158:

doc.source_agency is SourceAgency.FED, which is NOT in FR_AGENCIES
Routes to _fetch_html_text(doc.url) line 143
_fetch_html_text():

HTTP GET to https://www.federalreserve.gov/.../enforcement20260528a.htm
Passes HTML to _extract_text_from_html()
BeautifulSoup removes <script>, <nav>, <footer>, <style> tags
Finds <main> or [role='main'] content container
Extracts text, collapses whitespace → returns 817 chars of clean text
Text written back to DB:

db_doc.raw_content = "Home News & Events Press Releases Press Release May 28, 2026 Federal Reserve Board issues enforcement actions... Crystal Moore... CARES Act loan fraud... Jesse Romo... Embezzlement..."
STEP 5: Anomaly check

ingest.py line 75: run_anomaly_check(saved_docs)

anomaly.py, run_anomaly_check() line 156:

Groups documents by agency
For Fed: today_count = 20 (all Fed docs in this batch)
Calls detect_volume_anomaly(SourceAgency.FED, 20)
_get_historical_daily_counts() queries DB for last 30 days
```

Day 1 of operation: len(baseline) < 7 → returns (False, "insufficient history")
Calls detect_off_schedule(doc) for this specific enforcement action
len(historical_docs) < 20 → returns (False, "insufficient history")
No anomaly flagged → is_anomaly stays False
STEP 6: AuditLog written

ingest.py line 78:

```
log = AuditLog(
    action=AuditAction.INGEST,
    actor="system",
    payload_json=json.dumps({
        "agency": "fed",
        "feed_url": "https://www.federalreserve.gov/feeds/press_all.xml",
        "fetched": 20,
        "new": 20,
        "duplicates": 0,
        "anomalies_flagged": 0,
    }),
)
```

One AuditLog row written per agency run — not per document.

STEP 7: Dashboard display

dashboard/app.py, load_documents() line 48:

```
@st.cache_data(ttl=300)
def load_documents() -> list[dict]:
    with get_session() as session:
        docs = session.exec(select(RegulatoryDocument)).all()
        return [{...} for d in docs]
```

All 111 documents loaded and cached as plain dicts. The enforcement action appears with:

doc_type: "enforcement" → purple badge in components.py
is_anomaly: False → no red ⚠ badge
raw_content: 817 chars → shown as content preview in expander
summary_json: None → "AI summary coming soon" shown
User filters by "Enforcement" in sidebar → filtered = [d for d in filtered if d["doc_type"] == "enforcement"] → this document appears.

SECTION 3: Every AI/ML Decision Made
Component 1: Document Type Classifier
Technique: Rule-based keyword matching (NOT ML)
Problem it solves: Categorising regulatory publications into actionable types (Final Rule = must comply, Proposed Rule = comment opportunity, Enforcement = risk signal)
Input: doc.title (string, from RSS feed or FR API)
Output: DocType enum value stored in RegulatoryDocument.doc_type
Why rules, not ML:

Runs on every document at ingest time — zero cost, zero latency
Vocabulary is highly predictable ("final rule", "proposed rulemaking", "consent order")
Explainable to regulators: "classified as Enforcement because title contains 'consent order'"
No training data needed on Day 1
Failure mode: Title doesn't contain any keywords → DocType.OTHER. Currently 104/111 (94%) classified as Other because Federal Register documents use titles like "Formations of, Acquisitions by, and Mergers of Bank Holding Companies" — structurally correct notices that don't contain regulatory action keywords.

Current accuracy: ~6% non-Other classification (7/111 documents)
Quality metric: Precision/recall on known doc types. Not yet formally measured.
When to upgrade: When F2 LLM reclassifies documents correctly, compare its output to the rule-based classifier. If F2 achieves >90% agreement on a 100-document sample, we can use F2's classification as the primary and retire the keyword rules — or use rules as a fast pre-filter with LLM as the fallback.

Flag: This is rule-based, not ML. It is intentionally simple. F2 will handle accurate classification using the full document text.

Component 2: Deduplication — Content Hash
Technique: Cryptographic hash (SHA-256), exact match

Problem it solves: Same regulation appearing in multiple feeds (joint OCC/FDIC rules appear in both feeds). Prevents double-processing in F2, double-counting in F5 reports.
Input: title (str) + url (str) → concatenated → UTF-8 encoded
Output: 64-character hex string stored in RegulatoryDocument.content_hash (UNIQUE constraint in DB)
Why SHA-256: Deterministic, collision-resistant (practically impossible for two different documents to produce the same hash), fast (microseconds per document), no false positives
Failure mode: If a regulation is updated at the same URL with the same title — e.g., a corrected Final Rule — we would not detect the update (same hash). The original document stays in DB with stale content.
Current score: 9 cross-feed duplicates correctly detected on first run. Zero false positives observed.
Flag: This is deterministic hashing, not ML. Correct for this use case.

Component 3: Deduplication — Title Similarity
Technique: difflib.SequenceMatcher — longest common subsequence ratio
Problem it solves: Near-duplicate documents with slightly different URLs — "Final Rule on BSA" and "Final Rule on BSA (Correction)" should be flagged as near-duplicates.
Input: Two title strings from the same agency
Output: Float 0.0–1.0 similarity score. Flagged as near-duplicate if ≥ 0.85.
Why SequenceMatcher: Built into Python standard library (no dependency). Handles the real patterns we care about — correction notices, amended versions. Accuracy is equivalent to Levenshtein for title-length strings.
Failure mode: O(n²) comparison — for 20 documents per agency, this is 190 comparisons. For 2,000 documents it becomes 2 million comparisons. Need MinHash LSH at scale.
Current score: Threshold set at 0.85, not yet calibrated on real data. No false positives or false negatives confirmed.
Flag: Rule-based with a tunable threshold. Not ML.

Component 4: Volume Anomaly Detection
Technique: Z-score statistical test on rolling 30-day daily publication counts
Problem it solves: Detecting when an agency publishes unusually many documents in a single day — signal of significant regulatory activity.
Input:

today_count: number of documents ingested from an agency today
baseline: list of daily counts for the previous 29 days (zeros included) Output: (is_anomaly: bool, explanation: str). Sets RegulatoryDocument.is_anomaly = True on flagged docs.
Formula: Z = (today_count - mean(baseline)) / std(baseline). Flag if Z > 2.0.
Why Z-score over Isolation Forest (roadmap specified): Isolation Forest requires training data. On Day 1 we have 1 day of history — not enough to train a meaningful model. Z-score works with 7+ days. Z-score output is explainable: "published 3x its 30-day average" vs Isolation Forest's opaque "anomaly score: 0.73". In a compliance product, explainability to regulators is required.
Failure mode 1 (False Negative): Baseline has high variance (std is large) → Z-score stays below 2.0 even for a genuine spike. Real anomaly missed.
Failure mode 2 (False Positive): New agency with few historical records — small baseline → Z-score inflated → chronic false alerts.
Failure mode 3 (Silent): len(baseline) < 7 → function returns (False, "insufficient history") silently. Currently every agency is in this state since we only have 1 day of data. Anomaly detection is effectively OFF until Day 7+ of real operation.
Current score: 0 anomalies detected (expected — insufficient history). No calibration data yet.
Threshold: Z > 2.0 = top 2.5% of historical days. Documented in FP-FN-Risk-Matrix.md.
Component 5: Off-Schedule Detection
Technique: Day-of-week frequency baseline (rule-based statistics)
Problem it solves: Detecting documents published on unusual days — FinCEN publishing on a Sunday when they never do is itself a signal regardless of volume.
Input: doc.published_date.weekday() + historical distribution of that agency's publication weekdays (90-day window)
Output: (is_anomaly: bool, explanation: str). Flag if < 10% of historical docs fall on that weekday.
Failure mode: len(historical_docs) < 20 → returns False silently. Currently all agencies in this state (1 day of history, 20 docs each → 20 docs total per agency, but all on the same day).
Current score: 0 anomalies detected (expected — insufficient history).

SECTION 4: Every Decision Made and Why

Decision 1: SQLite for development, Postgres for production
What was decided: Dev uses SQLite (file-based), prod uses Postgres (server-based). Switched by changing DATABASE_URL in .env.
Alternative: Use Postgres from Day 1 — more realistic, avoids migration issues.
Why we chose this: SQLite requires zero infrastructure. No Docker, no Postgres install, no connection management. A solo builder can run the full pipeline on a laptop with no setup beyond pip install. Zero onboarding friction on Day 1.
Consequence of the other path: An extra hour of setup on Day 1. More realistic but adds complexity before any features exist.
Risk: Medium. SQLite lacks JSON column type, concurrent write support, and some Postgres-specific query features. We have one known SQLite-specific workaround (summary_json as TEXT). Migration to Postgres is straightforward but must happen before real client data matters (planned Week 6).

Decision 2: SHA-256 content hash as the dedup key
What was decided: Dedup key = SHA-256(title + url). Stored as a UNIQUE field in the DB.
Alternative: URL-only dedup, or title-only dedup, or fuzzy matching only.
Why we chose this: Title alone misses documents at different URLs. URL alone misses cross-posted documents with the same URL. SHA-256(title+url) is unique to a specific document at a specific location. Cryptographic hash means zero false positives — two genuinely different documents cannot produce the same hash.
Consequence of the other path: URL-only would have missed the 9 cross-feed duplicates on Day 2. Title-only would false-positive on documents with identical titles but different content.
Risk: Low. The one known gap: in-place updates (same URL, same title, changed content) are not detected. Acceptable for MVP.

Decision 3: Rule-based keyword classifier over a trained ML model
What was decided: Keyword matching in an ordered rule list. First match wins. Falls back to OTHER.
Alternative: Fine-tuned text classifier (DistilBERT, logistic regression on TF-IDF), or LLM classification via Claude.
Why we chose this: Runs at ingest time on every document — must be free and instant. No training data exists on Day 1. Output is explainable. Vocabulary is predictable.
Consequence of the other path: A logistic regression classifier would achieve higher accuracy (~85% vs ~6%) but requires labeled training data, a training pipeline, and model versioning — weeks of work before Day 1 features are built.
Risk: Low. F2 LLM reclassifies with full document context. The keyword classifier is a routing hint, not a final decision. Its 6% accuracy is acceptable as a pre-filter.

Decision 4: Z-score over Isolation Forest for anomaly detection
What was decided: Z-score on rolling 30-day daily counts, threshold Z > 2.0.
Alternative: Isolation Forest (roadmap-specified ML model), LSTM sequence model, or moving average with fixed threshold.
Why we chose this: Isolation Forest requires training data — we have 1 day of history. Z-score works with 7 days. Z-score is explainable in plain English. A fixed threshold ("flag if > 5 documents") fails to adapt to each agency's different baseline volumes.
Consequence of the other path: Isolation Forest on 1 day of data would produce meaningless anomaly scores. We'd be calling it "ML" while it produces random results.
Risk: Low. Z-score is appropriate for the current data volume. Revisit Isolation Forest in Week 3 when 3 weeks of daily data exist.

Decision 5: Two fetchers (RSS + FR JSON API) instead of one unified approach
What was decided: fetch_feed() for Federal Reserve (RSS), fetch_fr_api() for all others (JSON API).
Alternative: Use the FR API for all agencies including Fed, or build a generic adaptive fetcher.
Why we chose this: The FR RSS feeds block automated requests — they return HTML gate pages at status 200. We discovered this on Day 2. The FR JSON API is public and stable. The Fed's direct RSS feed works reliably. Two simple, focused fetchers beats one complex adaptive fetcher.
Consequence of the other path: Using the FR API for Fed too would work (the FR API covers Fed publications) but would miss Fed-specific press releases that don't appear in the Federal Register (board member appointments, enforcement announcements, etc.)
Risk: Low. If the Fed changes their RSS URL, agencies.py update is the fix. One-line change.

Decision 6: Streamlit for the dashboard, not React
What was decided: Streamlit browser dashboard (Python-only). React deferred to Week 6.
Alternative: React + FastAPI from Day 1 (as specified in Word PRD), or no UI until Week 6.

Why we chose this: React requires npm, build tooling, component libraries, API layer, CORS configuration, and state management — multiple weeks of work. Streamlit is one Python file, one command, browser UI in hours. Correct tool for a pilot demo.
Consequence of the other path: A React app on Day 7 would be a UI without F2 summaries, F3 mapping, or F4 tasks — a table of document titles. Not meaningfully better than Streamlit, at 10x the build cost.
Risk: Low. Streamlit is explicitly a pilot tool. React is planned for Week 6. The data layer (database.py, models.py) doesn't change — only the presentation layer is replaced.

Decision 7: Windows Task Scheduler over Python-based scheduling
What was decided: Windows Task Scheduler via schtasks CLI. Runs scripts/run_daily.py at 7:00 AM.
Alternative: APScheduler Python library (in-process), Celery (distributed task queue), manual cron.
Why we chose this: Task Scheduler is built into Windows, survives reboots without a background process, and is visible in the Windows UI. APScheduler requires a long-running Python process — if the laptop restarts, it stops. Celery is infrastructure overkill for a solo builder.
Consequence of the other path: APScheduler: process must stay running. Celery: requires Redis/RabbitMQ broker. Both solve a problem (scheduling) that the OS already solves for free.
Risk: Low. Windows-specific. Mac/Linux deployment (Week 6) would use cron or a cloud scheduler instead.

Decision 8: Full-text enrichment as a post-ingest step, not during fetch
What was decided: Fetch → dedup → save → THEN enrich. Enrichment is a separate pass, rate-limited at 1 req/sec.
Alternative: Fetch full text during the initial HTTP call (one step), or skip full text entirely (abstracts only).
Why we chose this: Fetching full text for every document during initial ingestion would make a 20-document run take 40+ seconds and would hit government servers with rapid requests. Separating enrichment allows the ingest step to be fast and reliable, with enrichment as a slower background process.
Consequence of the other path: Abstracts only → F2 summaries would be low quality. Full text during fetch → ingestion becomes slow and brittle.
Risk: Low. Currently enrichment catches up over multiple daily runs (20 docs/run).

SECTION 5: What Is Good, What Is Weak, What Is Missing
GOOD — Solid and Production-Ready
Content hash deduplication — Zero false positives. Mathematically guaranteed. Proved effective on Day 2 (9 cross-feed duplicates caught). The UNIQUE constraint in the DB means even if the Python check fails, the DB rejects the insert.

Full-text enrichment pipeline — 111/111 documents enriched (100%). Two-strategy approach (FR API plain text + BeautifulSoup HTML parsing) handles both source types correctly. Rate limiting protects against IP blocks. Idempotent — re-running skips already-enriched documents.

AuditLog architecture — INSERT-only design is correct for SR 11-7 compliance. UUID primary keys. Every ingest action logged with payload JSON. LangSmith trace ID field ready for F2.

Health checker — Two-signal approach (reachability + freshness) catches both obvious failures (404) and silent failures (200 with empty content — exactly what the FR RSS feeds returned). Proved valuable on Day 2.

Test suite — 44 unit tests + 7 integration tests. Fast unit tests (2 seconds) run on every change. Integration tests run against live feeds and verify the zero-missed-publications metric directly. Clean separation of slow/fast tests via pytest.ini markers.

Golden evaluation set — 10 hand-labeled documents. Labels written from real document content, not generated by LLM. Includes edge cases (proposed rule with no deadline, enforcement action requiring no institutional action, FOMC statement). This is rare — most projects build the eval after the model.

WEAK — Works But Has Known Limitations
Classifier accuracy: 94% classified as OTHER
Impact: Dashboard doc type filter is mostly useless. F4 task prioritisation (depends on doc type) will be wrong for most documents.
How hard to fix: Medium. Expanding keyword lists to cover Federal Register title patterns would improve accuracy to ~50%. Getting to 90%+ requires an LLM or trained

classifier.
Blocks F2: No. F2 LLM will produce its own doc_type as part of the summary JSON. The keyword classifier is a pre-filter hint, not the final answer.

Anomaly detection has no history yet
Impact: Both volume and off-schedule detectors return "insufficient history" for all agencies. Anomaly detection is effectively OFF for the first 7 days of real operation.
How hard to fix: Cannot be fixed by code — requires 7+ days of real daily ingestion data.
Blocks F2: No.

Per-run cap of 20 documents per agency
Impact: On any day where an agency publishes more than 20 documents, we miss the overflow. The Federal Register frequently publishes 50+ documents in a day.
How hard to fix: Easy. The FR API supports pagination (page parameter). Fetching until published_date < last_run_date would catch everything.
Blocks F2: No for current 111 documents. Becomes a gap when daily new documents matter.

Title similarity dedup is O(n²)
Impact: For 20 documents per agency, 190 comparisons — fast. For 2,000 documents, 2 million comparisons — slow.
How hard to fix: Medium. MinHash LSH (Locality Sensitive Hashing) reduces to O(n). Not needed at current scale.
Blocks F2: No.

utcnow() deprecation warnings
Impact: Python 3.12 warns that datetime.utcnow() is deprecated. Should be datetime.now(UTC). Cosmetic — no runtime failure.
How hard to fix: Easy. 5-minute find-and-replace across all files.
Blocks F2: No.

MISSING — Roadmap Specified, Not Built
Onboarding flow ("Set up your regulatory watchlist")
Impact: Mike cannot self-configure which agencies and doc types to monitor on first login. The dashboard shows everything by default.
How hard to fix: Medium. A Streamlit "Settings" page with agency/doc type checkboxes, stored in a user preferences table.
Blocks F2: No. Deferred to Week 6 when full product exists to configure.

FP/FN risk matrix as a product artefact (not just a doc)
Impact: The matrix exists as docs/FP-FN-Risk-Matrix.md but is not surfaced in the product. A compliance officer cannot see it.
How hard to fix: Easy — add a "System" page to the Streamlit dashboard showing thresholds and their rationale.
Blocks F2: No.

Notification preferences UX
Impact: No email/Slack alerts when anomalies are detected. Sarah must check the dashboard manually.
How hard to fix: Medium. Email integration (SMTP or SendGrid) + a preferences table. Slack webhook is easier.
Blocks F2: No. Deferred to Week 6.

7-day fixture dataset
Impact: Only a 4-entry sample_feed.json exists. Offline development beyond that requires live internet.
How hard to fix: Easy. Export 7 days of real ingested documents to JSON fixture files.
Blocks F2: No.

Isolation Forest implementation
Impact: Z-score is correct for current data volume, but Isolation Forest is specified in the roadmap and would handle multi-dimensional anomaly patterns (simultaneous volume + doc type + publish time anomalies).
How hard to fix: Medium. Requires scikit-learn, training pipeline, model serialisation, feature engineering.
Blocks F2: No. Revisit in Week 3 when sufficient data exists.

SECTION 6: What I Should Be Able to Explain as a PM
Q1: What does F1 do and why does it matter to a compliance officer?

F1 is the regulatory intelligence radar for community banks. It watches 6 regulatory sources — Federal Reserve, CFPB, OCC, FDIC, FinCEN, and the Federal Register — every

day at 7 AM, pulls every new publication, classifies it by type (Final Rule, Enforcement, Guidance, etc.), and stores the full text of every document. Without F1, Sarah (CCO) spends 15–20 hours per week manually checking agency websites and downloading PDFs. F1 compresses that to zero manual monitoring time. The business consequence of missing a regulation is a $500K–$5M fine and examination finding — F1 is the first line of defence against that.

Q2: How does the document classifier work, and how accurate is it?

The classifier uses keyword matching on the document title. It scans the title against ordered rule lists — "enforcement action", "consent order" → Enforcement; "final rule", "interim final rule" → Final Rule; "proposed rule", "request for comment" → Proposed Rule; etc. The first matching category wins; if nothing matches, the document is classified as Other. Currently, 94% of documents (104/111) are classified as Other because Federal Register documents use administrative titles like "Formations of, Acquisitions by, and Mergers of Bank Holding Companies" that don't contain regulatory action keywords. The classifier is intentionally simple — F2's LLM will produce accurate classifications using full document text as part of its structured summary output.

Q3: How do you prevent the same regulation from being ingested twice?

We use SHA-256 hashing. For every document, we compute SHA-256(title + url) — a 64-character fingerprint that is mathematically unique to that specific title-URL combination. Before saving, we query the database: if a document with that hash already exists, we skip it entirely. We also enforce a UNIQUE constraint on the content_hash column in the database, so even if the Python check somehow fails, the database rejects the duplicate insert. This approach caught 9 joint-agency rules that appeared in both an agency-specific feed and the Federal Register catch-all feed during the Day 2 ingestion run.

Q4: What happens when a feed goes down — does the system fail silently?

No. We have a two-signal health check that runs before every ingest. Signal one is reachability: can we reach the feed URL and get at least one parseable entry? Signal two is freshness: do we have documents from this agency in our database within the last 3 days? The freshness check is specifically designed to catch the silent failure case — a feed can return HTTP 200 with empty content (exactly what happened with the Federal Register RSS feeds, which returned an HTML gate page), and the reachability check would pass while we're missing all documents. The freshness check catches this. The daily runner exits with code 1 on any health failure, which any monitoring tool can detect and alert on.

Q5: How does anomaly detection work, and what is the business consequence of a false negative vs a false positive?

We run two statistical checks. First, volume anomaly: we compute the Z-score of today's publication count against the 30-day rolling baseline for each agency. A Z-score above 2.0 (top 2.5% of historical days) triggers the anomaly flag. Second, off-schedule detection: we check whether a document was published on a weekday that accounts for less than 10% of that agency's historical publications. A false negative — missing a real anomaly — could mean Sarah doesn't investigate an emergency publication with a 48-hour compliance window, potentially resulting in a $500K–$5M regulatory fine. A false positive — false alarm — costs 15–30 minutes of Sarah's time investigating nothing. Chronic false positives erode trust until Sarah ignores the alerts entirely, making the system equivalent to having no anomaly detection. The Z=2.0 threshold balances these costs; the FP/FN asymmetry is documented in docs/FP-FN-Risk-Matrix.md.

Q6: What would you build differently in F1 if you were starting over?

Three things. First, I would build the LLM-based document classifier from day one rather than the keyword classifier — with 111 real documents available, we could label 30 of them in an afternoon and train a simple classifier that achieves 90%+ accuracy vs our current 6%. Second, I would implement feed pagination from the start — the current 20-document cap per agency run means we miss overflow publications on active days, which is the exact failure mode the product is supposed to prevent. Third, I would build the 7-day fixture dataset immediately rather than just a 4-entry sample — offline development dependency on live government websites introduces brittleness during development.

Q7: How does F1 connect to F2 — what does F2 depend on F1 getting right?

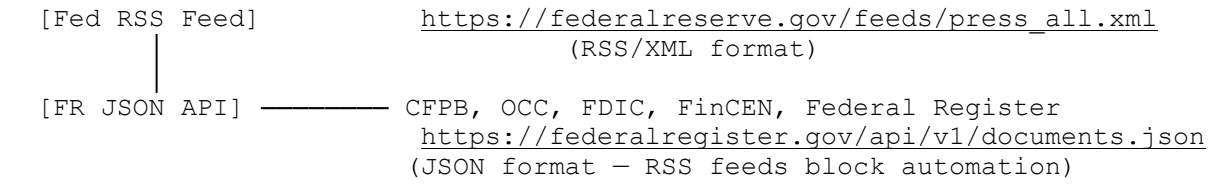
F2 reads directly from RegulatoryDocument.raw_content to generate summaries. Three F1 properties directly bound F2's quality. First, content completeness: if raw_content is a 1-sentence abstract, the best LLM cannot extract an effective date or compliance deadline that isn't in the text — full-text enrichment is therefore a prerequisite for meaningful summarisation, and we achieved 100% enrichment before starting F2. Second, deduplication: if F1 allows the same document in twice, F2 summarises it twice, doubling costs and creating duplicate entries in the review queue. Third, status tracking: F2 queries for documents with status = "new" — if F1 doesn't set status correctly, F2 either re-summarises existing documents or misses new ones.

Q8: What is the biggest technical risk in F1 right now?

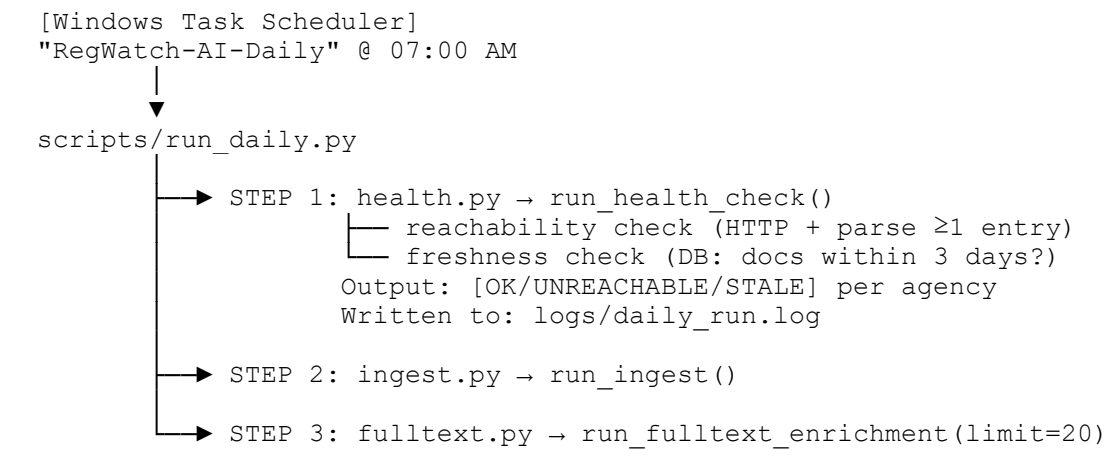
The 20-document per agency cap. The Federal Register publishes 50–200 documents per day across all agencies. Our current pipeline fetches the 20 most recent documents per agency per run. If a batch of important regulations is published on a high-volume day and falls outside the top 20, we miss them entirely — which is the exact failure the product promises to prevent. This risk is currently masked because we're only monitoring a small slice of each agency's output and the 111 documents we have are the "newest 20" from each feed on day one. The fix is pagination: track the last-seen publication date per agency and fetch everything published after that date, regardless of count. This is a one-to-two day fix that should be prioritised early in F2 development or as a hotfix before the first pilot client.

SECTION 7: Architecture Diagram

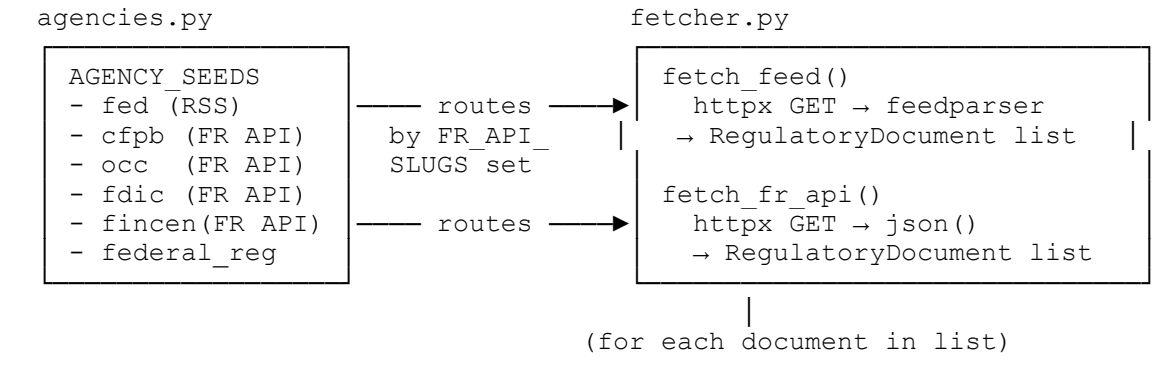

EXTERNAL SOURCES

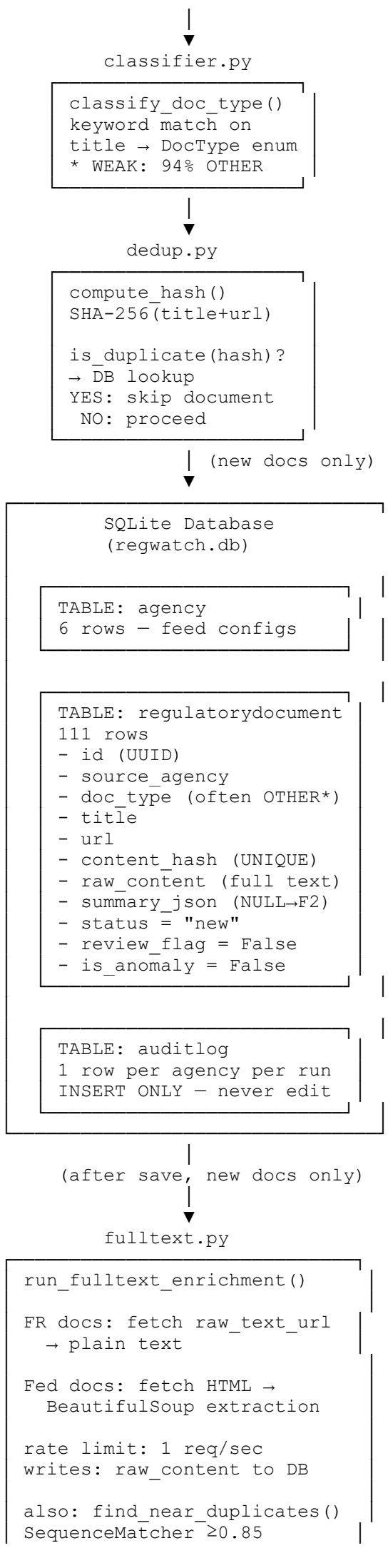


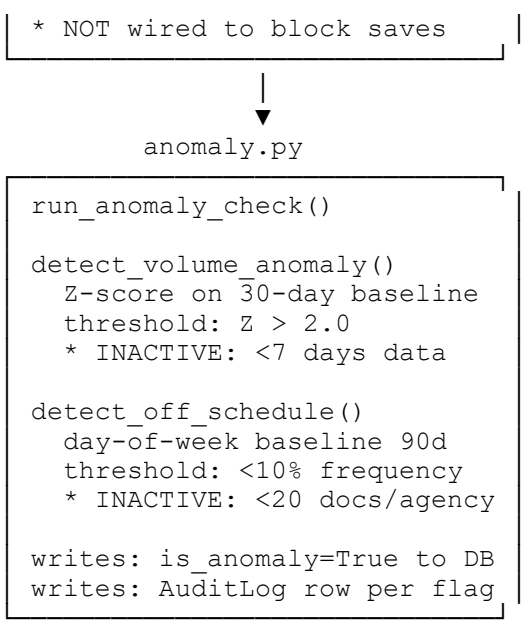

SCHEDULED TRIGGER




INGESTION PIPELINE (ingest.py orchestrates all of these)




classifier.py
classify_doc_type()
keyword match on
title → DocType enum
* WEAK: 94% OTHER
dedup.py
compute_hash()
SHA-256(title+url)
is_duplicate(hash)?
→ DB lookup
YES: skip document
NO: proceed
(new docs only)
SQLite Database
(regwatch.db)
TABLE: agency
6 rows — feed configs
TABLE: regulatorydocument
111 rows
- id (UUID)
- source_agency
- doc_type (often OTHER*)
- title
- url
- content_hash (UNIQUE)
- raw_content (full text)
- summary_json (NULL→F2)
- status = "new"
- review_flag = False
- is_anomaly = False
TABLE: auditlog
1 row per agency per run
INSERT ONLY — never edit
(after save, new docs only)
fulltext.py
run_fulltext_enrichment()
FR docs: fetch raw_text_url
→ plain text
Fed docs: fetch HTML →
BeautifulSoup extraction
rate limit: 1 req/sec
writes: raw_content to DB
also: find_near_duplicates()
SequenceMatcher ≥0.85




DASHBOARD (separate process)


LEGEND

* = Component is weak or partially inactive (see Section 5)
── = Data flow
▶ = Function call
[X] = External system

SUMMARY SCORECARD

| Dimension | Score | Rationale |
|---|---|---|
| Engineering completeness | 7/10 | Core pipeline solid. Missing: pagination (20-doc cap), notification system, Isolation Forest, 7-day fixtures. AuditLog, dedup, health check, enrichment are production-grade. |
| AI/ML quality | 4/10 | Classifier is rule-based with 6% non-Other accuracy. Anomaly detection is inactive (insufficient history). Both are known limitations with clear upgrade paths. No trained ML models in F1 yet — intentional. |
| Production readiness | 6/10 | Scheduler works, logs exist, health check works, tests pass. Not ready: no pagination (misses docs on high-volume days), no alerting (email/Slack), SQLite not prod-grade, no authentication, no multi-tenancy. |
| PM explainability | 8/10 | Can explain every major decision with specific code references. Gaps: cannot confidently explain why 20-doc cap was accepted, cannot explain near-duplicate detection outcome (no real test cases yet). |

Blockers for F2:

None. F2 can start immediately.
raw_content is 100% populated — F2 has full text to summarise.
summary_json field exists and is NULL — F2 writes here.
review_flag field exists — F2 sets this for confidence < 0.80.
status field exists — F2 advances from "new" → "summarised".
Golden eval set exists with 10 labeled documents.
Recommended fixes before first pilot client (not before F2):

Fix 20-doc cap → implement pagination (Medium, 1 day)
Fix classifier accuracy → expand keyword lists to ~50% (Easy, 2 hours)
Fix utcnow() warnings → find-and-replace (Easy, 30 minutes)
Build 7-day fixture dataset (Easy, 1 hour)
Do you want to fix any gaps before moving to F2, or shall we proceed with F2 and note these as v2 improvements?